



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
RAI TECHNOLOGY UNIVERSITY

Estd. under Karnataka Act. No. 40 of 2013

Assignment No: 02

Subject Title: Deep Learning

Subject Code: 0CS351T

Submitted By:

Student Name: Tejas Kumar H

University Enrolment Number: RTU24101IT012

Program Name: B. TECH IT AIML

Semester: 5th Sem

Session: 2025-2026

Submitted To:

Faculty Name: Mr. Sathyam

Department of Computer Science & Engineering

College Of Engineering

Rai Technology University

MNIST Handwritten Digit Classification Using the TensorFlow/Keras.

Aim:- To Build a Simple neural network Using the MNIST Handwritten digit dataset and TensorFlow/Keras to Classify Handwritten digits from 0 to 9, Evaluate its Performance, Visualize Training results, and Compare the model after Changing our hyper parameters.

Objective:-

- Load And Explore the MNIST dataset.
- Display Sample Handwritten digit images
- Build a neural Network Using TensorFlow/Keras
- Train the Model On Hand written digits
- Evaluate the model Using test accuracy.
- Visualize training and Validation accuracy and loss.
- Test the model on Hand written images.
- Compare actual and Predicted labels.

- Perform an experiment by Changing the number of neurons and Compare the results.

3. Theory

MNIST Stands for Modified National Institute Standards & Technology. It contains gray scale images of handwritten digits from 0 to 9.

Each image has Resolution of:

28 X 28 Pixels

The Dataset Contains:

- 60,000 Images.
- 10,000 Images
- 10 Classes: 0, 1, 2, ..., 9

A Neural Network learns Patterns from the Pixel Values and Uses those Patterns to Predict the digit represented by an Image.

Neural Network Architecture

The model Used in this assignment is:

Image



Flatten ($28 \times 28 \rightarrow 784$)



Dense Layer - 128 neurons, ReLU



Dense Layer - 64 neurons, ReLU



Output Layer - 10 neurons, Softmax

ReLU

The ReLU activation function is:

$$f(x) = \max(0, x)$$

It introduces non-linearity into neural Network of this.

Softmax

The Output layer Uses Softmax to convert the network outputs into Probability for the 10 Digit (Classes)

4. Requirements:

Install the Required Libraries:

Pip install tensorflow matplotlib numpy

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
from tensorflow.keras.datasets import mnist
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Flatten, Dense, Dropout
```

```
Print("TensorFlow Version:", tf.__version__)
```

```
# Load Dataset
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Print("\n Dataset Information")
```

```
Print(" Training Images:", x_train.shape)
```

```
Print(" Training Labels:", y_train.shape)
```

```
Print(" Training Images:", x_test.shape)
```

```
Print(" Testing Labels:", y_test.shape)
```

```
# Explore Dataset
```

```
Print("\n Pixel Value range:")
```

```
Print(" Maximum:", x_train.min())
```

```
Print(" Maximum:", x_train.max())
```

```
Print("\n First 10 Labels:")
```

```
Print(y_train[0:10])
```

```
# Display Sample Images
```

```
plt.figure(figsize=(10, 5))
```

```
for i in range(10):
```

```
    plt.subplot(2, 5, i+1)
```

```
    plt.imshow(x_train[i], cmap="gray")
```

```
plt.title("Label:" + str(y_train[i]))  
plt.axis("off")
```

```
plt.subplot("Sample MNIST Images")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Normalize data
```

```
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
```

```
# Build Baseline Model.
```

```
model = Sequential([
```

```
    Flatten(input_shape=(28, 28)),
```

```
    Dense(128, activation="relu"),
```

```
    Dense(64, activation="relu"),
```

```
    Dense(10, activation="relu"),
```

```
    Dense(10, activation="softmax")])
```

```
# Compile Model
```

→

```
model.compile(  
    optimizer = "adam",  
    loss = "SparseCategoricalCrossentropy",  
    metrics = ["accuracy"]  
)
```

```
print("\n Model Summary")
```

```
model.summary()
```

```
history = model.fit(  
    X_train,  
    Y_train,  
    epochs=10,  
    batch_size=128,  
    validation_split=0.2,  
    verbose=1  
)
```

```
# Evaluate Model
```

```
test_loss, test_accuracy = model.evaluate(  
    X_test,  
    Y_test,  
    verbose=0  
)
```

```
Print("In Test Results")
```

```
Print(" Test Loss:", test_loss)
```

```
Print(" Test Accuracy:", test_accuracy)
```

```
# Visualize Accuracy
```

```
Plt.figure Size=(figSize=(8,5))
```

```
Plt.plot(  
    history.history["accuracy"],  
    label="Training Accuracy"  
)
```

```
Plt.plot(  
    history.history["val_accuracy"],  
    label="Validation Accuracy"  
)
```

```
Plt.title("Training and Validation Accuracy")
```

```
Plt.xlabel("Epoch")
```

```
Plt.ylabel("Accuracy")
```

```
Plt.legend()
```

```
Plt.grid(True)
```

```
Plt.Show()
```

```
# Visualize loss
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
    history.history["loss"],  
    label = "Training Loss"  
)
```

```
plt.plot(  
    history.history["val_loss"],  
    label = "Validation Loss"  
)
```

```
plt.title("Training and Validation Loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.grid(True)
```

```
plt.show()
```

```
Predictions = model(x_test[:5])
```

```
Predicted_label = np.argmax(predictions, axis=1)
```

```
Print("\n Five True Predictions")
```

```

for i in range(5):
    print(
        "image", i+1,
        "\n predicted:", Predicted_labels[i]
    )
# Display 5 Test Images
plt.figure(figsize=(10,4))

for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(X_test[i], cmap="gray")
    plt.title(
        f" Actual: {X_test[i]} \n"
        f" Predicted: {Predicted_labels[i]}"
    )
    plt.axis("off")

plt.subplots("Actual vs Predicted Labels")
plt.tight_layout()
plt.show()

```

Observation:

The Experiment Increases the Capacity of the neural network by Using more neurons. This can allow the model to learn more complex patterns, but it also increases the number of parameters and the Computational Requirements.

Result:-

The MNIST hand written digit Classification System was Successfully Implemented Using the tensor flow / keras

The neural network was trained to classify the hand written digits from 0 to 9. Training and the Validation accuracy/loss were Visualized, and the trained model was tested on unseen images

Test Accuracy: 97.65%.

Conclusion :

A Simple feed forward neural network was Successfully developed for hand written digit Classification Using the MNIST dataset. The model learned the patterns from 28×28 gray scale images is Classified from into ten digit classes. Training and Validation Curves provide a Visual Understanding of model learning, while testing on Unseen Images demonstrated the model's Prediction Capability.